

The Gray-code Graph of t -ary Trees Using RD-sequences Is Hamiltonian*

Ro-Yu Wu¹ Jou-Ming Chang² Yue-Li Wang³

¹ Department of Industrial Engineering Management, Lunghwa University of Science and Technology, Taoyuan, Taiwan. (eric@mail.lhu.edu.tw)

² Department of Information Management, National Taipei College of Business, Taipei, Taiwan. (spade@mail.ntcb.edu.tw)

³ Department of Computer Science and Information Engineering, National Chi Nan University, Nantou, Taiwan. (yuelwang@ncnu.edu.tw)

Abstract

Usually, t -ary trees are encoded to integer sequences as their representation. It has been shown that the right-distance sequences (or RD-sequence for short) are concise representations of t -ary trees and are suitable for the usage of loopless Gray-code enumeration. For $n, t \geq 2$, the RD-representation graph, denoted as RD_n^t , is a graph whose vertices are the RD-sequences of t -ary trees with n internal nodes, and two vertices are joined by an edge if and only if their RD-sequences differ in exactly one position. From the definition, a Gray-code enumeration for t -ary trees can be implemented by finding a Hamiltonian path in RD_n^t . In [24], the authors have proposed an algorithm to find a Hamiltonian path in RD_n^t . In this paper, we further determine some basic graph parameters of RD_n^t , including order, size, degrees of vertices, eccentricity, diameter, radius, center, and median. Moreover, we show that every RD-representation graph is Hamiltonian, i.e., it contains a simple cycle passing through all vertices of RD_n^t .

*This research was partially supported by National Science Council under the Grants NSC95-2221-E-260-024.

1. Introduction

An easy way to represent combinatorial objects is the use of integer sequences encoding. To generate objects efficiently, enumerating algorithms are postulated to run in a constant time for each generation under the worst case, and so that the number of changes between two consecutive sequences can be bounded by a constant. A recommendable choice for designing an enumerating algorithm is the use of a technique called loopless (i.e., algorithm involves no recursion and, after initialization, uses only “if-then-else” statements and assignments) [2, 23] and such that the output sequences result in a Gray-code order [18].

A *Gray-code graph* of objects is a graph containing all objects of the same size as its vertices and such that two vertices are connected by an edge if their corresponding sequences differ in exactly one position. Consequently, an implementation of Gray-code enumeration for objects can be done by finding a Hamiltonian path in the Gray-code graph. Till now, many researches have been devoted to the loopless Gray-code enumeration of bi-

nary trees and their generalization (i.e., t -ary trees with $t \geq 2$) using diverse integer sequences [6, 15, 20–22, 26, 27]. However, Lucas [7] mentioned that the Gray-code graph of n -node binary trees has a less regular structure and a more complicated induction is required to prove the existence of a Hamiltonian cycle, i.e., a simple cycle passes through all vertices of the graph.

As a substitute for the study of Gray-code graphs of binary trees, Lucas [7] applied a similar concept to the following definition. A *rotation graph* of n -node binary trees, denoted as $RG(n)$, is a graph consisting of all n -node binary trees as its vertices and such that two vertices are joined by an edge if their corresponding binary trees can be transformed to each other by using a single rotation (i.e., an operation used in the balance of binary trees). As a consequence, the existence of a Hamiltonian cycle in $RG(n)$ has already been proved in [7]. For further results related to rotation graphs and their structure properties, we can also refer to [3, 8, 12–14, 16, 17, 19].

Similarly, Korsh and Lipschutz [5] introduced an operation called *shift* on t -ary trees. A shift is a particular operation that transforms a t -ary tree into another and preserves their preorder numbers of vertices. Then, they defined a graph to be a *shift graph* if it contains all n -node t -ary trees as its vertices and an edge in the graph corresponds to one shift operation. By finding a Hamiltonian path in the shift graph, they presented a loopless algorithm to generate all n -node t -ary trees with a constant time between two successive generations.

In this paper, we use a concise representation, called right-distance sequences (or RD-sequences for short), to describe t -ary trees with n internal nodes. In [24], the authors have shown the existence of a Hamiltonian path in the Gray-code graph of t -ary trees using RD-sequences. As a by-product, such a construction of Hamiltonian path can also

achieve a loopless Gray-code enumeration algorithm for t -ary trees. We now extend the result to show that every Gray-code graph of t -ary trees using RD-sequences is hamiltonian.

2. RD-sequences and RD-representation Graphs

A t -ary tree T , $t \geq 2$, is an ordered tree defined recursively as follows: T is either empty or it has a distinguished node r , called the root, that connects to subtrees $T_1, T_2, \dots, T_t$, where each subtree is a t -ary tree. Let $\mathcal{T}_{n,t}$ be the set consisting of t -ary trees with n internal nodes. From [4], we have known that the number of t -ary trees with n internal nodes is

$$\begin{aligned} |\mathcal{T}_{n,t}| &= \frac{1}{tn+1} \binom{tn+1}{n} \\ &= \frac{1}{(t-1)n+1} \binom{tn}{n}. \end{aligned} \quad (1)$$

Usually, t -ary trees are encoded to integer sequences as their representation. For each $T \in \mathcal{T}_{n,t}$, the path from the root to its rightmost node is called the right arm of T . We assume that all internal nodes are numbered from 1 to n in preorder (i.e. visit recursively the root and then the subtrees of T from left to right). Then, we can represent T by using the following definition.

Definition 1 For each internal node $i \in T$, the *right-distance* d_i is defined as follows:

$$d_i = \begin{cases} 0 & \text{if } i \text{ is on the right arm of } T; \\ d_{p(i)} + t - k & \text{otherwise,} \end{cases}$$

where $p(i)$ is the parent of i , and i is the k th child (from left to right) of $p(i)$ with $1 \leq k \leq t$. The resulting sequence, denoted by $rd(T) = (d_1, d_2, \dots, d_n)$, is called the *right-distance sequence* (or RD-sequence for short) of T .

Note that $d_i \leq d_{i-1} + (t-1)$ and $d_i = d_{p(i)}$ when i is the rightmost child of its parent. We

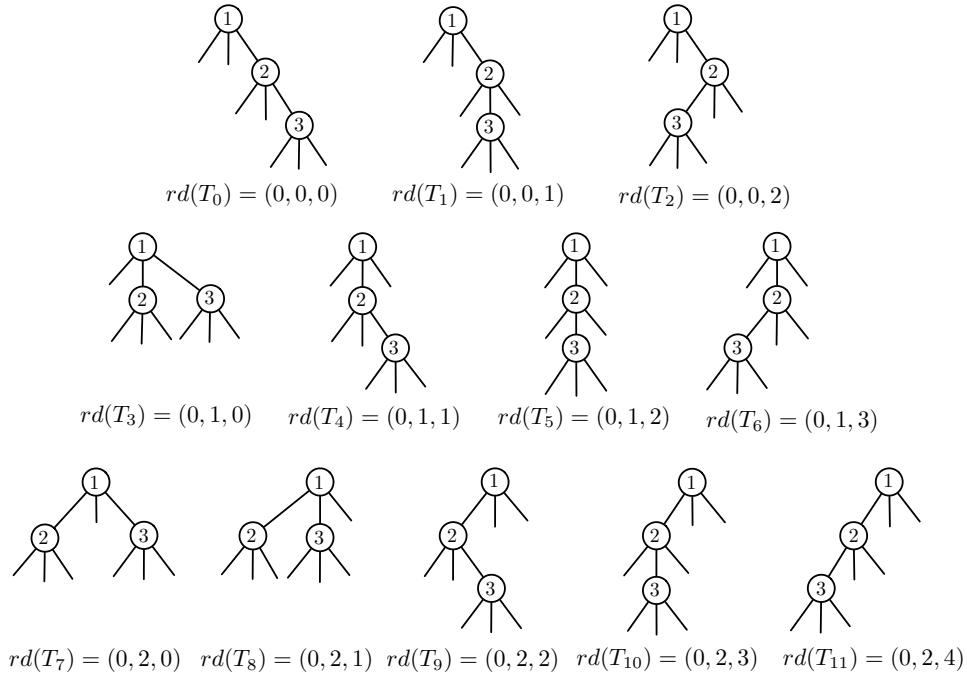


Figure 1: The ternary trees with 3 internal nodes.

remark that RD-sequence is also called the *inversion table* due to [11] and [1]. However, we use the more apposite term “RD-sequence” which is similar to the representation introduced by Mäkinen [9, 10] for measuring the distance of nodes in binary trees. By (1), there are twelve ternary trees for $n = 3$. Figure 1 depicts their RD-sequences.

To describe all t -ary trees in $\mathcal{T}_{n,t}$ using RD-sequences, a systematical way is defined as follows.

Definition 2 Given an integer $t \geq 2$, the t -ary recursion tree with n levels (or simply call *recursion tree* if t and n are clear) is a rooted tree $\mathbb{T}$ constructed by the following rules: (1) Every node in $\mathbb{T}$ is associated with a label; (2) The first level contains a single node (i.e., the root) with label 0; and (3) Every nonleaf node (i.e., node is not in the n th level) with label k has $t + k$ children labeled by $0, 1, \dots, t + k - 1$ from left to right in the next level.

According to the definition, the full labels along the path from the root to a leaf in $\mathbb{T}$ can represent the RD-sequence of a t -ary tree with n internal nodes. Figure 2(a) demonstrates the ternary recursion tree with three levels. Similar concepts of recursion trees were independently introduced under various names for other representation of integer sequences, including the backtracking tree [28], the infinite tree [11], the recursion tree [8] for $t = 2$ (i.e., the case of binary trees), and the coding tree [25] for $t \geq 2$.

Definition 3 For $n, t \geq 2$, the *Gray-code graph* is a graph such that every vertex is corresponding to the representation sequence of a t -ary tree with n internal nodes, and two vertices are joined by an edge if and only if their representation sequences differ in exactly one position. In particular, the *RD-representation graph*, denoted as RD_n^t , is a Gray-code graph using RD-sequences as its representation.

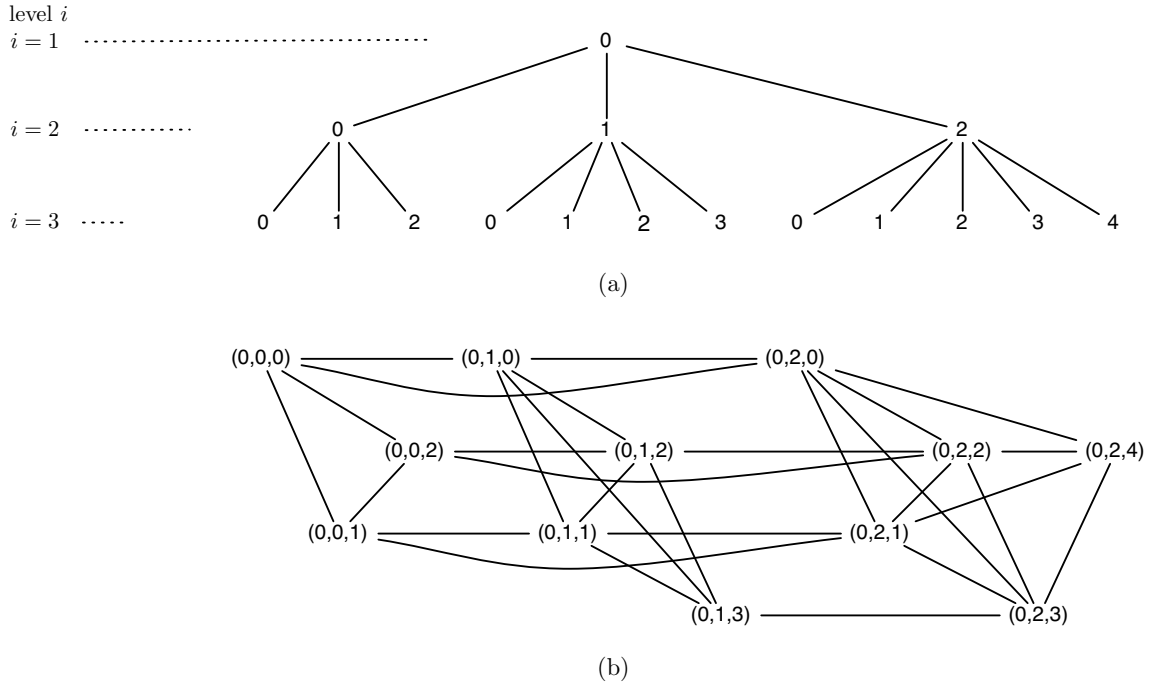


Figure 2: (a) The ternary recursion tree $\mathbb{T}$ for $n = 3$; (b) The graph RD_3^3 .

For example, the graph RD_3^3 is shown in Figure 2(b). From the above definition, it is easy to see that a Gray-code enumeration for t -ary trees using RD-sequences can be implemented by finding a Hamiltonian path in RD_n^t and then listing the labels of vertices along the path. To generate RD-sequences in Gray-code order, Wu et al. [24] recently propose a loopless algorithm with worst-case constant time per each sequence. As a remark, the proposed algorithm is designed to arrange the nodes in each level of $\mathbb{T}$ in a particular order and without really building the recursion tree and its corresponding RD-representation graph. The structure of RD_n^t is interesting because it has been proved in [24] that the number of Hamiltonian paths in RD_n^t is at least 2^{n-1} . In the next section, we will show that RD_n^t for $n, t \geq 2$ is indeed Hamiltonian.

In what follows, we give some graph parameters of RD_n^t . Clearly, for any vertex $u \in RD_n^t$, there exists another vertex $v \in RD_n^t$ such that the RD-sequences of u and v differ in $n-1$ positions. Thus, the eccentricity of u ,

the diameter of RD_n^t , and the radius of RD_n^t are equal to $n-1$. Consequently, all vertices of RD_n^t are in the center (i.e., the set of vertices whose eccentricity equals to the radius). Also, the set $\{(0, t-1, t-1, \dots, t-1, k) \mid 0 \leq k \leq t-1\}$ induces a clique of size t in RD_n^t and forms the median (i.e., the set of vertices with the minimum sum of distances to all other vertices) of RD_n^t . For instance, the median of RD_3^3 is $\{(0, 2, 0), (0, 2, 1), (0, 2, 2)\}$.

For a vertex $v \in RD_n^t$ with RD-sequence $rd(T_v) = (d_1, d_2, \dots, d_n)$, the degree of v can be computed recursively as follow:

$$\deg(d_1, d_2, \dots, d_n) \quad (2)$$

$$= \begin{cases} \deg(d_1, d_2, \dots, d_{n-1}) + (t-1) + d_{n-1} & \text{if } n \geq 2 \text{ and } d_n = 0, 1, \dots, t-1; \\ \deg(d_1, d_2, \dots, d_{n-1}) + (t-1) + d_{n-1} - k & \text{if } n \geq 2 \text{ and } d_n = (t-1) + k \text{ for } k = 1, 2, \dots, d_{n-1}; \\ 0 & \text{if } n = 1. \end{cases}$$

For example, $\deg(0, 2, 3) = \deg(0, 2) + (3 - 1) + 2 - 1 = \deg(0) + (3 - 1) + 0 + 3 = 5$ in RD_3^3 . In particular, the minimum degree and the maximum degree of RD_n^t can be computed by formulae $\delta(RD_n^t) = (t - 1)(n - 1)$ and $\Delta(RD_n^t) = (t - 1)(2n - 3)$, respectively. Clearly, an RD-representation graph has order $p = |\mathcal{T}_{n,t}|$, and from (2), the size q can be determined by

$$q = \frac{1}{2} \sum_{v \in RD_n^t} \deg(rd(T_v)).$$

Thus, applying an enumerating algorithm proposed in [24] to generate all RD-sequences of t -ary trees with n internal nodes, we list the mentioned parameters of RD_n^3 for small n in Table 1.

Table 1: The minimum degree, maximum degree, order, and size of RD_n^3

n	$\delta(RD_n^3)$	$\Delta(RD_n^3)$	p	q
1	0	0	1	0
2	2	2	3	3
3	4	6	12	29
4	6	10	55	222
5	8	14	273	1575
6	10	18	1428	10812

3. Construction of Hamiltonian Cycles

By arranging the nodes in each level of $\mathbb{T}$ in a particular order, Wu et al. [24] have already demonstrated the existence of a Hamiltonian path in RD_n^t . The following is the sketch of their algorithm. For every nonleaf node x with label k in $\mathbb{T}$, all the labels of $t + k$ children of x are permuted in increasing order (i.e., $1, 2, \dots, k + t - 1, 0$) or decreasing order (i.e., $0, k + t - 1, \dots, 2, 1$) from left to right. Then, the two types of permutations are alter-

nately appeared in each level of $\mathbb{T}$. For example, Figure 3(a) shows the final arrangement of $\mathbb{T}$ for $t = 2$ and $n = 5$. It is easy to see that the list of full labels in the resulting recursion tree from left to right is in a Gray-code order. Hence, we can obtain a corresponding Hamiltonian path in RD_5^2 which is shown in Figure 3(b). In this figure, we merely label d_5 instead of $(d_1, d_2, d_3, d_4, d_5)$ to represent a vertex for simplicity, and some edges joining vertices are omitted in the drawing for clarity.

We are now in a position to show our main result.

Theorem 1 *For $n, t \geq 2$, there exists a Hamiltonian cycle in RD_n^t .*

Proof. We shall present a way to construct a Hamiltonian cycle in RD_n^t . We first observe that a set of vertices $(d_1, d_2, \dots, d_{n-1}, d_n)$ with only the difference at d_n forms a maximal clique in RD_n^t . Since the i th level of $\mathbb{T}$ contains $|\mathcal{T}_{i,t}|$ nodes, there are $|\mathcal{T}_{n-1,t}|$ maximal cliques in RD_n^t . We use $K_0, K_1, \dots, K_{|\mathcal{T}_{n-1,t}|-1}$ to denote these cliques according to the ordering in the arrangement of $\mathbb{T}$ suggested by [24]. Let (i, K_j) stand for the vertex $(d_1, d_2, \dots, d_{n-1}, i)$ if it belongs to the maximal clique K_j . For each K_j , since $t \geq 2$, K_j contains at least two vertices. We define C_{K_j} as the cycle $(0, K_j)(1, K_j) \cdots (|K_j| - 1, K_j)(0, K_j)$ in K_j . Thus, if $|K_j| = 2$ (this case occurs only when $t = 2$), then C_{K_j} contains two edges $(0, K_j)(1, K_j)$ and $(1, K_j)(0, K_j)$. For any two consecutive cycles C_{K_j} and $C_{K_{j+1}}$ defined above (where j and $j + 1$ are taken modulo $|\mathcal{T}_{n-1,t}|$), we combine these cycles in the order to form a Hamiltonian cycle according to the following conditions.

Case 1: $|C_{K_j}| = |C_{K_{j+1}}|$.

Remove the edge $(|K_j| - 1, K_j)(0, K_j)$ in C_{K_j} and the edge $(|K_j| - 1, K_{j+1})(0, K_{j+1})$ in $C_{K_{j+1}}$, respectively. And then, add the two edges $(|K_j| - 1, K_j)(|K_j| - 1, K_{j+1})$ and

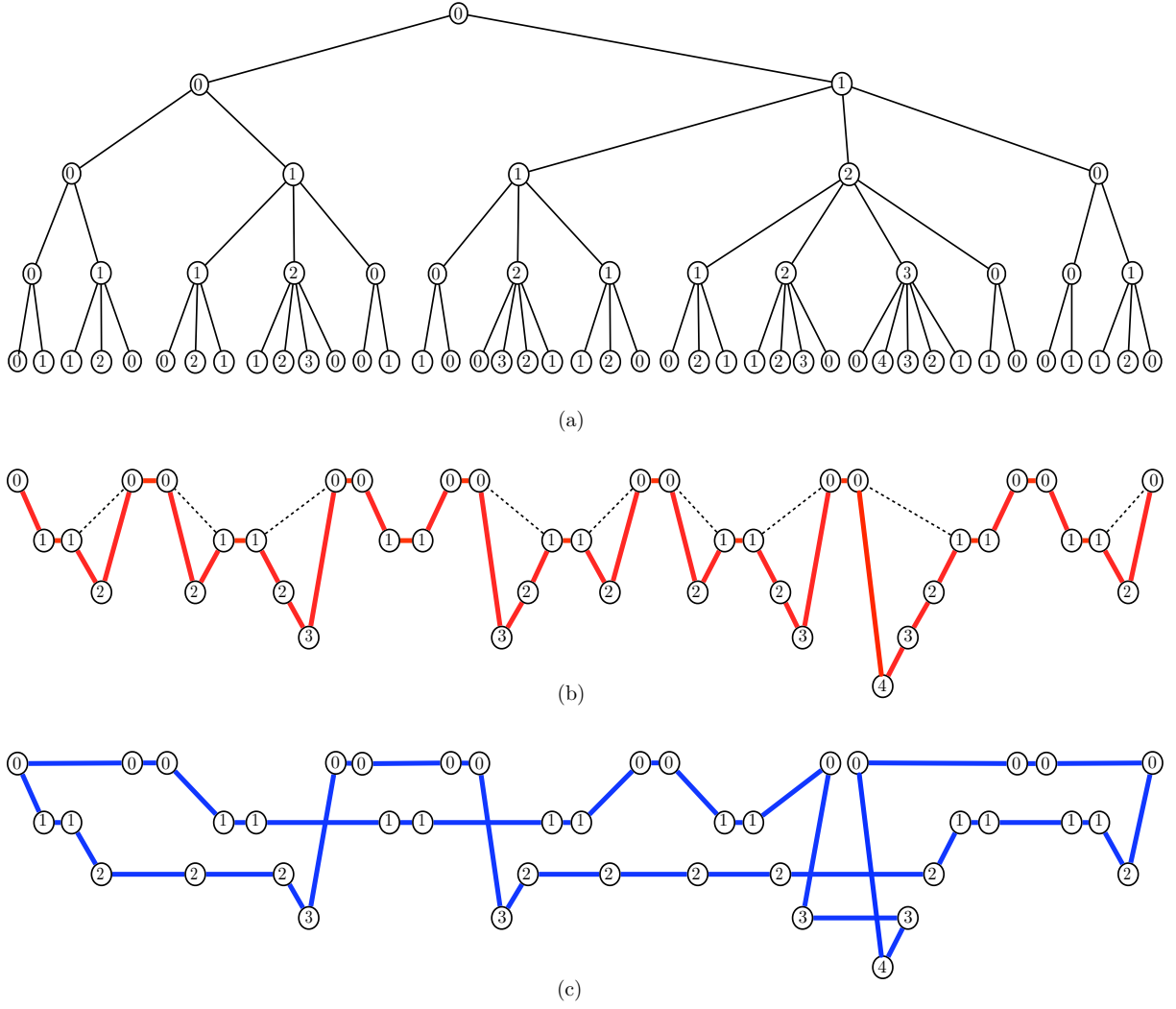


Figure 3: (a) The binary recursion tree for $n = 5$; (b) A Hamiltonian path in RD_5^2 ; (c) A Hamiltonian cycle in RD_5^2 .

$(0, K_j)(0, K_{j+1})$ between K_j and K_{j+1} . Hence, C_{K_j} and $C_{K_{j+1}}$ are combined into a larger cycle.

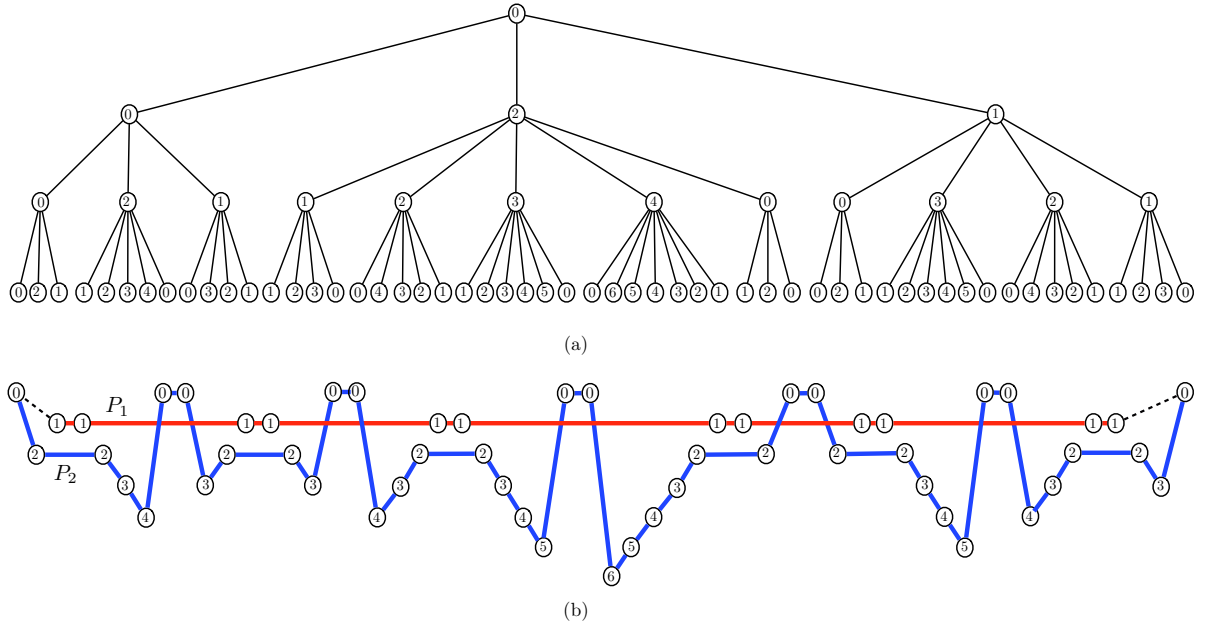
Case 2: $|C_{K_j}| < |C_{K_{j+1}}|$.

Remove the edge $(|K_j| - 2, K_j)(|K_j| - 1, K_j)$ in C_{K_j} and the edge $(|K_j| - 2, K_{j+1})(|K_j| - 1, K_{j+1})$ in $C_{K_{j+1}}$, respectively. And then, add the two edges $(|K_j| - 2, K_j)(|K_j| - 2, K_{j+1})$ and $(|K_j| - 1, K_j)(|K_j| - 1, K_{j+1})$ between K_j and K_{j+1} . Similarly, a larger cycle containing all vertices of C_{K_j} and $C_{K_{j+1}}$ is ob-

tained.

Case 3: $|C_{K_j}| > |C_{K_{j+1}}|$.

Remove the edge $(|K_{j+1}| - 2, K_j)(|K_{j+1}| - 1, K_j)$ in C_{K_j} and the edge $(|K_{j+1}| - 2, K_{j+1})(|K_{j+1}| - 1, K_{j+1})$ in $C_{K_{j+1}}$, respectively. And then, add the two edges $(|K_{j+1}| - 2, K_j)(|K_{j+1}| - 2, K_{j+1})$ and $(|K_{j+1}| - 1, K_j)(|K_{j+1}| - 1, K_{j+1})$ between K_j and K_{j+1} . Thus, C_{K_j} and $C_{K_{j+1}}$ are combined into a larger cycle.


 Figure 4: The ternary recursion tree for $n = 4$ and a Hamiltonian cycle of RD_4^3 .

In the above three cases, all adding edges are contained in RD_n^t because any two vertices connecting one of these edges differ by only one label at the first $n - 1$ positions in their RD-sequences. As a result, a Hamiltonian cycle of RD_n^t can be constructed (see an example in Figure 3(c) for illustration). $\square$

For $t \geq 3$, there is another way to achieve a Hamiltonian cycle in RD_n^t . We again use the arrangement of $\mathbb{T}$ suggested by [24] to order the maximal cliques of RD_n^t . Thus, two vertices with the same last label d_n in any two consecutive maximal cliques have exactly one difference at the first $n - 1$ positions in their RD-sequences. We let P_1 be the path passing through vertices with labels $(d_1, d_2, \dots, d_{n-1}, 1)$ along the successive maximal cliques. Next, we construct another path, called P_2 , that uses the remaining vertices of RD_n^t and is described as follows. All vertices in a maximal clique K consecutively occur in P_2 such that their last labels d_n 's appear in decreasing order (i.e., $0, |K| - 1, |K| - 2, \dots, 2$) or increasing order (i.e., $2, 3, \dots, |K| - 1, 0$),

and two such types of orderings alternately appeared in P_2 . Finally, we use two additional edges $(0, K_0)(1, k_0)$ and $(0, K_{|\mathcal{T}_{n-1,t}|-1})(1, K_{|\mathcal{T}_{n-1,t}|-1})$ to connect P_1 and P_2 to form a Hamiltonian cycle of RD_n^t . Figure 4 shows the recursion tree and the resulting Hamiltonian cycle of RD_4^3 .

References

- [1] S. G. Akl and I. Stojmenović, Generating t -ary trees in parallel, *Nordic Journal of Computing* 3 (1996) 63–71.
- [2] G. Ehrlich, Loopless algorithms for generating permutations, combination, and other combinatorial configurations, *Journal of the ACM* 20 (1973) 500–513.
- [3] F. Hurtado and M. Noy, Graph of triangulations of a convex polygon and tree of triangulations, *Computational Geometry* 13 (1999) 179–188.
- [4] D. E. Knuth, The Art of Computer Programming. Vol. 1: Fundamental Algorithms, Addison-Wesley, Reading, MA, 1968.

- [5] J. F. Korsh and S. Lipschutz, Shifts and loopless generation of k -ary trees, *Information Processing Letters* 65 (1998) 235–240.
- [6] J. F. Korsh and P. LaFollette, Loopless generation of Gray code for k -ary trees, *Information Processing Letters* 70 (1999) 7–11.
- [7] J. M. Lucas, The rotation graph of binary trees is Hamiltonian, *Journal of Algorithms* 8 (1987) 503–535.
- [8] J. M. Lucas, D. Roelants van Baronaigien, and F. Ruskey, On rotations and the generation of binary trees, *Journal of Algorithms* 15 (1993) 343–366.
- [9] E. Mäkinen, Left distance binary tree representations, *BIT* 27 (1987) 163–169.
- [10] E. Mäkinen, Generating random binary trees – A survey, *Information Sciences* 115 (1999) 123–136.
- [11] H. W. Martin and B. J. Orr, A random binary tree generator, in: *Proc. of the 17th conference on ACM Annual Computer Science Conference*, Louisville, Kentucky, February 21–23, 1989, pp. 33–38.
- [12] J. Pallo, Enumerating, ranking and unranking binary trees, *The Computer Journal* 29 (1986) 171–175.
- [13] J. Pallo, On the rotation distance in the lattice of binary trees, *Information Processing Letters* 25 (1987) 369–373.
- [14] J. Pallo, Some properties of the rotation lattice of binary trees, *The Computer Journal* 31 (1988) 564–565.
- [15] D. Roelants van Baronaigien, A loopless Gray-code algorithm for listing k -ary trees, *Journal of Algorithms* 35 (2000) 100–107.
- [16] R. O. Rogers and R. D. Dutton, Properties of the rotation graph of binary trees, *Congressus Numerantium* 109 (1995) 51–63.
- [17] R. O. Rogers and R. D. Dutton, On distance in the rotation graph of binary trees, *Congressus Numerantium* 120 (1996) 103–113.
- [18] C. D. Savage, A survey of combinatorial Gray codes, *SIAM Review* 39 (1997) 605–629.
- [19] D. D. Sleator, R. E. Tarjan, and W. R. Thurston, Rotation distance, triangulations and hyperbolic geometry, *Journal of the American Mathematical Society* 1 (1988) 647–681.
- [20] T. Takaoka, $O(1)$ time algorithms for combinatorial generation by tree traversal, *The Computer Journal* 42 (1999) 400–408.
- [21] V. Vajnovszki, On the loopless generation of binary tree sequences, *Information Processing Letters* 68 (1998) 113–117.
- [22] V. Vajnovszki, Generating a Gray code for P-sequences, *Journal of Mathematical Modelling and Algorithms* 1 (2002) 31–41.
- [23] S. G. Williamson, *Combinatorics for Computer Science*, Computer Science Press, Rockville, MD, 1985.
- [24] Ro-Yu Wu, Jou-Ming Chang, Yue-Li Wang, A loopless algorithm to generate t -ary trees using RD-sequences, submitted paper (The 24th Workshop on Combinatorial Mathematics and Computation Theory, 2007).
- [25] L. Xiang, K. Ushijima, and S. G. Akl, Generating regular k -ary trees efficiently, *The Computer Journal* 43 (2000) 290–300.
- [26] L. Xiang, K. Ushijima, and C. Tang, Efficient loopless generation of Gray codes for k -ary trees, *Information Processing Letters* 76 (2000) 169–174.
- [27] L. Xiang and K. Ushijima, On $O(1)$ time algorithms for combinatorial generation, *The Computer Journal* 44 (2001) 292–302.
- [28] D. Zerling, Generating binary trees using rotations, *Journal of Association for Computing Machinery* 32 (1985) 694–701.